

```

1 //#####
2 // FILE:   FinalProjectStarter_main.c
3 //
4 // TITLE:  Final Project Starter
5 //#####
6
7 // Included Files
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <stdarg.h>
11 #include <string.h>
12 #include <math.h>
13 #include <limits.h>
14 #include "F28x_Project.h"
15 #include "F2837xD_SWPrioritizedIsrLevels.h"
16 #include "driverlib.h"
17 #include "device.h"
18 #include "F28379dSerial.h"
19 #include "song.h"
20 #include "dsp.h"
21 #include "fpu32/fpu_rfft.h"
22 #include "xy.h"
23 #include "MatrixMath.h"
24 #include "SE423Lib.h"
25 #include "OptiTrack.h"
26
27 #define PI          3.1415926535897932384626433832795
28 #define TWOPI       6.283185307179586476925286766559
29 #define HALFPi      1.5707963267948966192313216916398
30 // The Launchpad's CPU Frequency set to 200 you should not change this value
31 #define LAUNCHPAD_CPU_FREQUENCY 200
32
33 // ----- code for CAN start here -----
34 #include "F28379dCAN.h"
35 // #define TX_MSG_DATA_LENGTH 4
36 // #define TX_MSG_OBJ_ID 0 //transmit
37
38 #define RX_MSG_DATA_LENGTH 8
39 #define RX_MSG_OBJ_ID_1 1 //measurement from sensor 1
40 #define RX_MSG_OBJ_ID_2 2 //measurement from sensor 2
41 #define RX_MSG_OBJ_ID_3 3 //quality from sensor 1
42 #define RX_MSG_OBJ_ID_4 4 //quality from sensor 2
43 // ----- code for CAN end here -----
44
45 // Interrupt Service Routines predefinition
46 __interrupt void cpu_timer0_isr(void);
47 __interrupt void cpu_timer1_isr(void);
48 __interrupt void cpu_timer2_isr(void);
49 __interrupt void SWI1_HighestPriority(void);
50 __interrupt void SWI2_MiddlePriority(void);
51 __interrupt void SWI3_LowestPriority(void);
52 __interrupt void ADC12_ISR(void);
53 __interrupt void SPIB_isr(void);
54 // ----- code for CAN start here -----
55 __interrupt void can_isr(void);
56 // ----- code for CAN end here -----
57
58 void setF28027EPWM1A(float controleffort);
59 int16_t EPwm1A_F28027 = 1500;
60 void setF28027EPWM2A(float controleffort);
61 int16_t EPwm2A_F28027 = 1500;
62
63 // ----- code for CAN start here -----
64 // volatile uint32_t txMsgCount = 0;
65 // extern uint16_t txMsgData[4];
66
67 volatile uint32_t rxMsgCount_1 = 0;
68 volatile uint32_t rxMsgCount_2 = 0;
69 extern uint16_t rxMsgData[8];
70
71 uint32_t dis_raw_1[2];
72 uint32_t dis_raw_2[2];
73 uint32_t dis_1 = 0;
74 uint32_t dis_2 = 0;
75
76 uint32_t quality_raw_1[4];
77 uint32_t quality_raw_2[4];
78 float quality_1 = 0.0;
79 float quality_2 = 0.0;
80
81 uint32_t lightlevel_raw_1[4];
82 uint32_t lightlevel_raw_2[4];
83 float lightlevel_1 = 0.0;
84 float lightlevel_2 = 0.0;
85
86 uint32_t measure_status_1 = 0;
87 uint32_t measure_status_2 = 0;
88
89 volatile uint32_t errorFlag = 0;
90 // ----- code for CAN end here -----
91
92 uint32_t numTimer0calls = 0;
93 uint16_t UARTPrint = 0;
94
95 float printLV1 = 0;
96 float printLV2 = 0;
97
98 float printLinux1 = 0;
99 float printLinux2 = 0;
100
101 uint16_t LADARi = 0;
102 char G_command[] = "G04472503\n"; //command for getting distance -120 to 120 degree
103 uint16_t G_len = 11; //length of command
104 xy_ladar_pts[228]; //xy data
105 float LADARrightfront = 0;
106 float LADARfront = 0;
107 float LADARtemp_x = 0;
108 float LADARtemp_y = 0;
109 extern datapts_ladar_data[228];
110
111 extern uint16_t newLinuxCommands;
112 extern float LinuxCommands[CMDNUM_FROM_FLOATS];

```

```

113
114 extern uint16_t NewLVData;
115 extern float fromLVvalues[LVNUM_TOFROM_FLOATS];
116 extern LVSendFloats_t DataToLabView;
117 extern char LVsenddata[LVNUM_TOFROM_FLOATS*4+2];
118 extern uint16_t LADARpingpong;
119 extern uint16_t NewCAMDataThreshold1; // Flag new data
120 extern float fromCAMvaluesThreshold1[CAMNUM_FROM_FLOATS];
121 extern uint16_t NewCAMDataThreshold2; // Flag new data
122 extern float fromCAMvaluesThreshold2[CAMNUM_FROM_FLOATS];
123
124 extern uint16_t NewCAMDataAprilTag1; // Flag new data
125 extern float fromCAMvaluesAprilTag1[CAMNUM_FROM_FLOATS];
126
127 float tagid = 0;
128 float tagx = 0;
129 float tagy = 0;
130 float tagz = 0;
131 float tagthetax = 0;
132 float tagthetay = 0;
133 float tagthetaz = 0;
134 int32_t numtag = 0;
135 int32_t AprilTagState = 10;
136 float KpAprilTag = -1.0;
137 float AprilTagSetPoint = 0;
138 int32_t AprilTagCount = 0;
139
140 float MaxAreaThreshold1 = 0;
141 float MaxColThreshold1 = 0;
142 float MaxRowThreshold1 = 0;
143 float NextLargestAreaThreshold1 = 0;
144 float NextLargestColThreshold1 = 0;
145 float NextLargestRowThreshold1 = 0;
146 float NextNextLargestAreaThreshold1 = 0;
147 float NextNextLargestColThreshold1 = 0;
148 float NextNextLargestRowThreshold1 = 0;
149
150 float MaxAreaThreshold2 = 0;
151 float MaxColThreshold2 = 0;
152 float MaxRowThreshold2 = 0;
153 float NextLargestAreaThreshold2 = 0;
154 float NextLargestColThreshold2 = 0;
155 float NextLargestRowThreshold2 = 0;
156 float NextNextLargestAreaThreshold2 = 0;
157 float NextNextLargestColThreshold2 = 0;
158 float NextNextLargestRowThreshold2 = 0;
159
160 float odist = 0.0;
161 float gdist = 0.0;
162
163 float kpvision = 0.0;
164 uint32_t count22 = 0;
165 uint32_t count1 = 0;
166
167 uint32_t numThres1 = 0;
168 uint32_t numThres2 = 0;
169
170 pose ROBOTps = {0,0,0}; //robot position
171 pose LADARps = {3.5/12,0,0,1}; // 3.5/12 for front mounting, theta is not used in this current code
172 float LADARxoffset = 0;
173 float LADARyoffset = 0;
174
175 uint32_t timecount = 0;
176 int16_t RobotState = 1;
177 int16_t checkfronttally = 0;
178 int32_t WallFollowtime = 0;
179
180
181 float colcentroid = 0.0;
182
183 #define NUMWAYPOINTS 10
184 uint16_t statePos = 0;
185 pose robotdest[NUMWAYPOINTS]; // array of waypoints for the robot
186 uint16_t i = 0; //for loop
187
188 uint16_t right_wall_follow_state = 2; // right follow
189 float Kp_front_wall = -2.0;
190 float front_turn_velocity = 0.2;
191 float left_turn_Stop_threshold = 3.5;
192 float Kp_right_wal = -4.0;
193 float ref_right_wall = 1.1;
194 float foward_velocity = 1.0;
195 float left_turn_Start_threshold = 1.3;
196 float turn_saturation = 2.5;
197
198 float x_pred[3][1] = {{0},{0},{0}}; // predicted state
199
200 //more kalman vars
201 float B[3][2] = {{1,0},{1,0},{0,1}}; // control input model
202 float u[2][1] = {{0},{0}}; // control input in terms of velocity and angular velocity
203 float Bu[3][1] = {{0},{0},{0}}; // matrix multiplication of B and u
204 float z[3][1]; // state measurement
205 float eye3[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // 3x3 identity matrix
206 float K[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // optimal Kalman gain
207 #define ProcUncert 0.0001
208 #define CovScalar 10
209 float Q[3][3] = {{ProcUncert,0,ProcUncert/CovScalar},
210                 {0,ProcUncert,ProcUncert/CovScalar},
211                 {ProcUncert/CovScalar,ProcUncert/CovScalar,ProcUncert}}; // process noise (covariance of encoders and gyro)
212 #define MeasUncert 1
213 float R[3][3] = {{MeasUncert,0,MeasUncert/CovScalar},
214                 {0,MeasUncert,MeasUncert/CovScalar},
215                 {MeasUncert/CovScalar,MeasUncert/CovScalar,MeasUncert}}; // measurement noise (covariance of OptiTrack Motion Capture measurement)
216 float S[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance
217 float S_inv[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance matrix inverse
218 float P_pred[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // predicted covariance (measure of uncertainty for current position)
219 float temp_3x3[3][3]; // intermediate storage
220 float temp_3x1[3][1]; // intermediate storage
221 float ytilde[3][1]; // difference between predictions
222
223 // Optitrack Variables
224 int32_t OptiNumUpdatesProcessed = 0;
225 pose OPTITRACKps;
226 extern uint16_t new_optitrack;

```

```

227 extern float Optitrackdata[OPTITRACKDATASIZE];
228 int16_t newOPTITRACKpose=0;
229
230 int16_t adcc2result = 0;
231 int16_t adcc3result = 0;
232 int16_t adcc4result = 0;
233 int16_t adcc5result = 0;
234 float adcC2Volt = 0.0;
235 float adcC3Volt = 0.0;
236 float adcC4Volt = 0.0;
237 float adcC5Volt = 0.0;
238 int32_t numADCCcalls = 0;
239 float LeftWheel = 0;
240 float RightWheel = 0;
241 float LeftWheel_1 = 0;
242 float RightWheel_1 = 0;
243 float LeftVel = 0;
244 float RightVel = 0;
245 float uLeft = 0;
246 float uRight = 0;
247 float HandValue = 0;
248 float vref = 0;
249 float turn = 0;
250 float gyro9250_drift = 0;
251 float gyro9250_angle = 0;
252 float old_gyro9250 = 0;
253 float gyroLPR510_angle = 0;
254 float gyroLPR510_offset = 0;
255 float gyroLPR510_drift = 0;
256 float old_gyroLPR510 = 0;
257 float gyro9250_radians = 0;
258 float gyroLPR510_radians = 0;
259
260 int16_t readdata[25];
261 int16_t IMU_data[9];
262
263 float accelx = 0;
264 float accely = 0;
265 float accelz = 0;
266 float gyrox = 0;
267 float gyroy = 0;
268 float gyroz = 0;
269
270 // Needed global Variables
271 float accelx_offset = 0;
272 float accely_offset = 0;
273 float accelz_offset = 0;
274 float gyrox_offset = 0;
275 float gyroy_offset = 0;
276 float gyroz_offset = 0;
277 int16_t calibration_state = 0;
278 int32_t calibration_count = 0;
279 int16_t doneCal = 0;
280
281 #define MPU9250 1
282 #define DAN28027 2
283 int16_t CurrentChip = MPU9250;
284 int16_t DAN28027Garbage = 0;
285 int16_t dan28027adc1 = 0;
286 int16_t dan28027adc2 = 0;
287 uint16_t MPU9250ignoreCNT = 0; //This is ignoring the first few interrupts if ADCC_ISR and start sending to IMU after these first few interrupts.
288
289 float RCangle;
290
291 void main(void)
292 {
293     // PLL, WatchDog, enable Peripheral Clocks
294     // This example function is found in the F2837xD_SysCtrl.c file.
295     InitSysCtrl();
296
297     InitGpio();
298
299     InitSE423DefaultGPIO();
300
301     //Scope for Timing
302     GPIO_SetupPinMux(11, GPIO_MUX_CPU1, 0);
303     GPIO_SetupPinOptions(11, GPIO_OUTPUT, GPIO_PUSHPULL);
304     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
305
306     GPIO_SetupPinMux(32, GPIO_MUX_CPU1, 0);
307     GPIO_SetupPinOptions(32, GPIO_OUTPUT, GPIO_PUSHPULL);
308     GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
309
310     GPIO_SetupPinMux(52, GPIO_MUX_CPU1, 0);
311     GPIO_SetupPinOptions(52, GPIO_OUTPUT, GPIO_PUSHPULL);
312     GpioDataRegs.GPBCLEAR.bit.GPIO52 = 1;
313
314     GPIO_SetupPinMux(60, GPIO_MUX_CPU1, 0);
315     GPIO_SetupPinOptions(60, GPIO_OUTPUT, GPIO_PUSHPULL);
316     GpioDataRegs.GPBCLEAR.bit.GPIO60 = 1;
317
318     GPIO_SetupPinMux(61, GPIO_MUX_CPU1, 0);
319     GPIO_SetupPinOptions(61, GPIO_OUTPUT, GPIO_PUSHPULL);
320     GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
321
322     GPIO_SetupPinMux(67, GPIO_MUX_CPU1, 0);
323     GPIO_SetupPinOptions(67, GPIO_OUTPUT, GPIO_PUSHPULL);
324     GpioDataRegs.GPCCLEAR.bit.GPIO67 = 1;
325
326 // ----- code for CAN start here -----
327 //GPIO17 - CANRXB
328 GPIO_SetupPinMux(17, GPIO_MUX_CPU1, 2);
329 GPIO_SetupPinOptions(17, GPIO_INPUT, GPIO_ASYNC);
330
331 //GPIO12 - CANTXB
332 GPIO_SetupPinMux(12, GPIO_MUX_CPU1, 2);
333 GPIO_SetupPinOptions(12, GPIO_OUTPUT, GPIO_PUSHPULL);
334 // ----- code for CAN end here -----
335
336
337
338 // ----- code for CAN start here -----
339 // Initialize the CAN controller
340 InitCANB();

```

```

341 // Set up the CAN bus bit rate to 1000 kbps
342 setCANBitRate(200000000, 1000000);
343
344
345 // Enables Interrupt line 0, Error & Status Change interrupts in CAN_CTL register.
346 CanbRegs.CAN_CTL.bit.IE0= 1;
347 CanbRegs.CAN_CTL.bit.EIE= 1;
348 // ----- code for CAN end here -----
349
350 // Clear all interrupts and initialize PIE vector table:
351 // Disable CPU interrupts
352 DINT;
353
354 // Initialize the PIE control registers to their default state.
355 // The default state is all PIE interrupts disabled and flags
356 // are cleared.
357 // This function is found in the F2837xD_PieCtrl.c file.
358 InitPieCtrl();
359
360 // Disable CPU interrupts and clear all CPU interrupt flags:
361 IER = 0x0000;
362 IFR = 0x0000;
363
364 // Initialize the PIE vector table with pointers to the shell Interrupt
365 // Service Routines (ISR).
366 // This will populate the entire table, even if the interrupt
367 // is not used in this example. This is useful for debug purposes.
368 // The shell ISR routines are found in F2837xD_DefaultIsr.c.
369 // This function is found in F2837xD_PieVect.c.
370 InitPieVectTable();
371
372 // Interrupts that are used in this example are re-mapped to
373 // ISR functions found within this project
374 EALLOW; // This is needed to write to EALLOW protected registers
375 PieVectTable.TIMER0_INT = &cpu_timer0_isr;
376 PieVectTable.TIMER1_INT = &cpu_timer1_isr;
377 PieVectTable.TIMER2_INT = &cpu_timer2_isr;
378 PieVectTable.SCIA_RX_INT = &RXAINT_recv_ready;
379 PieVectTable.SCIB_RX_INT = &RXBINT_recv_ready;
380 PieVectTable.SCIC_RX_INT = &RXCINT_recv_ready;
381 PieVectTable.SCID_RX_INT = &RXDINT_recv_ready;
382 PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
383 PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
384 PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
385 PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
386 PieVectTable.ADCC1_INT = &ADCC_ISR;
387 PieVectTable.SPIB_RX_INT = &SPIB_isr;
388 PieVectTable.EMIF_ERROR_INT = &SWI1_HighestPriority; // Using Interrupt12 interrupts that are not used as SWIs
389 PieVectTable.RAM_CORRECTABLE_ERROR_INT = &SWI2_MiddlePriority;
390 PieVectTable.FLASH_CORRECTABLE_ERROR_INT = &SWI3_LowestPriority;
391 // ----- code for CAN start here -----
392 PieVectTable.CANB0_INT = &can_isr;
393 // ----- code for CAN end here -----
394 EDIS; // This is needed to disable write to EALLOW protected registers
395
396 // Initialize the CpuTimers Device Peripheral. This function can be
397 // found in F2837xD_CpuTimers.c
398 InitCpuTimers();
399
400 // Configure CPU-Timer 0, 1, and 2 to interrupt every second:
401 // 200MHz CPU Freq, Period (in uSeconds)
402 ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000); // Currently not used for any purpose
403 ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 100000); // !!!!! Important, Used to command LADAR every 100ms. Do not Change.
404 ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 40000); // Currently not used for any purpose
405
406 // Enable CpuTimer Interrupt bit TIE
407 CpuTimer0Regs.TCR.all = 0x4000;
408 CpuTimer1Regs.TCR.all = 0x4000;
409 CpuTimer2Regs.TCR.all = 0x4000;
410
411 DELAY_US(1000000); // Delay 1 second giving Lidar Time to power on after system power on
412
413 init_serialSCIB(&SerialB,19200);
414 init_serialSCIC(&SerialC,19200);
415
416 for (LADARi = 0; LADARi < 228; LADARi++) {
417     ladar_data[LADARi].angle = ((3*LADARi+44)*0.3515625-135)*0.01745329; //0.017453292519943 is pi/180, multiplication is faster; 0.3515625 is 360/1024
418 }
419
420 init_eQEPs();
421 init_EPWM1and2();
422 init_RCServoPWM_3AB_5AB_6A();
423 init_ADCsAndDACs();
424 setupSpib();
425
426 EALLOW;
427 EPwm4Regs.ETSEL.bit.SOCAEN = 0; // Disable SOCA on A group
428 EPwm4Regs.TBCTL.bit.CTRMODE = 3; // freeze counter
429 EPwm4Regs.ETSEL.bit.SOCASEL = 2; // Select Event when counter equal to PRD
430 EPwm4Regs.ETPS.bit.SOCAPRD = 1; // Generate pulse on 1st event
431 EPwm4Regs.TBCTR = 0x0; // Clear counter
432 EPwm4Regs.TBPHS.bit.TBPHS = 0x0000; // Phase is 0
433 EPwm4Regs.TBCTL.bit.PHSEN = 0; // Disable phase loading
434 EPwm4Regs.TBCTL.bit.CLKDIV = 0; // divide by 1 50Mhz Clock
435 EPwm4Regs.TBPRD = 50000; // Set Period to 1ms sample. Input clock is 50MHz.
436 // Notice here that we are not setting CMPA or CMPB because we are not using the PWM signal
437 EPwm4Regs.ETSEL.bit.SOCAEN = 1; //enable SOCA
438 //EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, wait to do this right before enabling interrupts
439 EDIS;
440
441 // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
442 // which is connected to CPU-Timer 1, and CPU int 14, which is connected
443 // to CPU-Timer 2: int 12 is for the SWI.
444 IER |= M_INT1;
445 IER |= M_INT6;
446 IER |= M_INT8; // SCIC SCID
447 IER |= M_INT9; // SCIA
448 IER |= M_INT12;
449 IER |= M_INT13;
450 IER |= M_INT14;
451
452 // Enable TINT0 in the PIE: Group 1 interrupt 7
453 PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
454 PieCtrlRegs.PIEIER1.bit.INTx3 = 1;

```

```

455 PieCtrlRegs.PIEIER6.bit.INTx3 = 1;
456 PieCtrlRegs.PIEIER12.bit.INTx9 = 1; //SWI1
457 PieCtrlRegs.PIEIER12.bit.INTx10 = 1; //SWI2
458 PieCtrlRegs.PIEIER12.bit.INTx11 = 1; //SWI3 Lowest priority
459 // ----- code for CAN start here -----
460 // Enable CANB in the PIE: Group 9 interrupt 7
461 PieCtrlRegs.PIEIER9.bit.INTx7 = 1;
462 // ----- code for CAN end here -----
463
464 // ----- code for CAN start here -----
465 // Enable the CAN interrupt signal
466 CanbRegs.CAN_GLB_INT_EN.bit.GLBINT0_EN = 1;
467 // ----- code for CAN end here -----
468
469
470 robotdest[0].x = -4; robotdest[0].y = 10;
471 robotdest[1].x = -4; robotdest[1].y = 2;
472 //middle of bottom
473 robotdest[2].x = 0; robotdest[2].y = 2;
474 //outside the course
475 robotdest[3].x = 0; robotdest[3].y = -3;
476 //back to middle
477 robotdest[4].x = 0; robotdest[4].y = 2;
478 robotdest[5].x = 4; robotdest[5].y = 2;
479 robotdest[6].x = 4; robotdest[6].y = 10;
480 robotdest[7].x = 0; robotdest[7].y = 9;
481 //cjca added two points
482 robotdest[8].x = 4; robotdest[8].y = 5;
483 robotdest[9].x = 2; robotdest[9].y = 8;
484
485 // ROBOTps will be updated by Optitrack during gyro calibration
486 // TODO: specify the starting position of the robot
487 ROBOTps.x = 0; //the estimate in array form (useful for matrix operations)
488 ROBOTps.y = 0;
489 ROBOTps.theta = 0; // was -PI: need to flip OT ground plane to fix this
490 x_pred[0][0] = ROBOTps.x; //estimate in structure form (useful elsewhere)
491 x_pred[1][0] = ROBOTps.y;
492 x_pred[2][0] = ROBOTps.theta;
493
494 AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
495 PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
496 SpibRegs.SPIFFRX.bit.RXFFOVFCLR=1; // Clear Overflow flag
497 SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
498 PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
499 EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, and enter up count mode
500
501 init_serialSCIA(&SerialA,115200);
502 init_serialSCID(&SerialD,2083332);
503
504 // Enable global Interrupts and higher priority real-time debug events
505 EINT; // Enable Global interrupt INTM
506 ERTM; // Enable Global realtime interrupt DBGM
507 // ----- code for CAN start here -----
508 // Measured Distance from 1
509 // Initialize the receive message object 1 used for receiving CAN messages.
510 // Message Object Parameters:
511 // Message Object ID Number: 1
512 // Message Identifier: 0x060b0101
513 // Message Frame: Standard
514 // Message Type: Receive
515 // Message ID Mask: 0x0
516 // Message Object Flags: Receive Interrupt
517 // Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
518 // for a Receive mailbox)
519 //
520 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_1, 0x060b0101, CAN_MSG_FRAME_EXT,
521 CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
522 RX_MSG_DATA_LENGTH);
523
524 // Measured Distance from 2
525 // Initialize the receive message object 2 used for receiving CAN messages.
526 // Message Object Parameters:
527 // Message Object ID Number: 2
528 // Message Identifier: 0x060b0102
529 // Message Frame: Standard
530 // Message Type: Receive
531 // Message ID Mask: 0x0
532 // Message Object Flags: Receive Interrupt
533 // Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
534 // for a Receive mailbox)
535 //
536
537 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_2, 0x060b0102, CAN_MSG_FRAME_EXT,
538 CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
539 RX_MSG_DATA_LENGTH);
540
541 // Measurement Quality from 1
542 // Initialize the receive message object 2 used for receiving CAN messages.
543 // Message Object Parameters:
544 // Message Object ID Number: 3
545 // Message Identifier: 0x060b0201
546 // Message Frame: Standard
547 // Message Type: Receive
548 // Message ID Mask: 0x0
549 // Message Object Flags: Receive Interrupt
550 // Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
551 // for a Receive mailbox)
552 //
553
554 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_3, 0x060b0201, CAN_MSG_FRAME_EXT,
555 CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
556 RX_MSG_DATA_LENGTH);
557
558 // Measurement Quality from 2
559 // Initialize the receive message object 2 used for receiving CAN messages.
560 // Message Object Parameters:
561 // Message Object ID Number: 4
562 // Message Identifier: 0x060b0202
563 // Message Frame: Standard
564 // Message Type: Receive
565 // Message ID Mask: 0x0
566 // Message Object Flags: Receive Interrupt
567 // Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
568 // for a Receive mailbox)

```

```

569 //
570
571 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_4, 0x060b0202, CAN_MSG_FRAME_EXT,
572                      CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
573                      RX_MSG_DATA_LENGTH);
574
575 //
576 // Start CAN module operations
577 //
578 CanbRegs.CAN_CTL.bit.Init = 0;
579 CanbRegs.CAN_CTL.bit.CCE = 0;
580
581
582 // ----- code for CAN end here -----
583 char S_command[19] = "S1152000124000\n"; // this change the baud rate to 115200
584 uint16_t S_len = 19;
585 serial_sendSCIC(&SerialC, S_command, S_len);
586
587
588 DELAY_US(1000000); // Delay letting Lidar change its Baud rate
589 init_serialSCIC(&SerialC, 115200);
590
591
592 // LED1 is GPIO22
593 GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
594 // LED2 is GPIO94
595 GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
596 // LED3 is GPIO95
597 GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
598 // LED4 is GPIO97
599 GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;
600 // LED5 is GPIO111
601 GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;
602
603
604 // IDLE loop. Just sit and loop forever (optional):
605 while(1)
606 {
607     if (UARTPrint == 1) {
608
609         if (readbuttons() == 0) {
610             UART_printfLine(1, "State: %d : %d", RobotState, statePos);
611             UART_printfLine(2, "0 %.3f G %.3f", odist, gdist);
612             //UART_printfLine(1, "Vrf: %.2f trn: %.2f", vref, turn);
613             //UART_printfLine(1, "x: %.2f y: %.2f a: %.2f", ROBOTps.x, ROBOTps.y, ROBOTps.theta);
614             UART_printfLine(2, "F%.4f R%.4f", LADARfront, LADARrightfront);
615         } else if (readbuttons() == 1) {
616             UART_printfLine(1, "01A: %.0fC: %.0fR: %.0f", MaxAreaThreshold1, MaxColThreshold1, MaxRowThreshold1);
617             UART_printfLine(2, "P1A: %.0fC: %.0fR: %.0f", MaxAreaThreshold2, MaxColThreshold2, MaxRowThreshold2);
618             //UART_printfLine(1, "LV1: %.3f LV2: %.3f", printLV1, printLV2);
619             //UART_printfLine(2, "Ln1: %.3f Ln2: %.3f", printLinux1, printLinux2);
620         } else if (readbuttons() == 2) {
621             UART_printfLine(1, "02A: %.0fC: %.0fR: %.0f", NextLargestAreaThreshold1, NextLargestColThreshold1, NextLargestRowThreshold1);
622             UART_printfLine(2, "P2A: %.0fC: %.0fR: %.0f", NextLargestAreaThreshold2, NextLargestColThreshold2, NextLargestRowThreshold2);
623             // UART_printfLine(1, "%.2f %.2f", adcC2Volt, adcC3Volt);
624             // UART_printfLine(2, "%.2f %.2f", adcC4Volt, adcC5Volt);
625         } else if (readbuttons() == 4) {
626             UART_printfLine(1, "03A: %.0fC: %.0fR: %.0f", NextNextLargestAreaThreshold1, NextNextLargestColThreshold1, NextNextLargestRowThreshold1);
627             UART_printfLine(2, "P3A: %.0fC: %.0fR: %.0f", NextNextLargestAreaThreshold2, NextNextLargestColThreshold2, NextNextLargestRowThreshold2);
628             // UART_printfLine(1, "L: %.3f R: %.3f", LeftVel, RightVel);
629             // UART_printfLine(2, "uL: %.2f uR: %.2f", uLeft, uRight);
630         } else if (readbuttons() == 8) {
631             UART_printfLine(1, "020x%.2f y%.2f", ladar_pts[20].x, ladar_pts[20].y);
632             UART_printfLine(2, "150x%.2f y%.2f", ladar_pts[150].x, ladar_pts[150].y);
633         } else if (readbuttons() == 3) {
634             UART_printfLine(1, "Vrf: %.2f trn: %.2f", vref, turn);
635             UART_printfLine(2, "MPU: %.2f LPR: %.2f", gyro9250_radians, gyroLPR510_radians);
636         } else if (readbuttons() == 5) {
637             UART_printfLine(1, "0x: %.2f 0y: %.2f 0a: %.2f", OPTITRACKps.x, OPTITRACKps.y, OPTITRACKps.theta);
638             UART_printfLine(2, "State: %d : %d", RobotState, statePos);
639         } else if (readbuttons() == 6) {
640             UART_printfLine(1, "D1 %ld D2 %ld", dis_1, dis_2);
641             UART_printfLine(2, "St1 %ld St2 %ld", measure_status_1, measure_status_2);
642         } else if (readbuttons() == 7) {
643             UART_printfLine(1, "%.0f, %.1f, %.1f, %.1f", tagid, tagx, tagy, tagz);
644             UART_printfLine(2, "%.1f, %.1f, %.1f", tagthetax, tagthetay, tagthetaz);
645         }
646         //UART_printfLine(1, "RCangle %.1f ", RCangle);
647
648         UARTPrint = 0;
649     }
650 }
651 }
652
653 __interrupt void SPIB_isr(void) {
654
655     uint16_t i;
656     GpioDataRegs.GPBSET.bit.GPIO32 = 1;
657
658     GpioDataRegs.GPCSET.bit.GPIO66 = 1; // Pull CS high as done R/W
659     GpioDataRegs.GPASET.bit.GPIO29 = 1; // Pull CS high for DAN28027
660
661     if (CurrentChip == MPU9250) {
662
663         for (i=0; i<8; i++) {
664             readdata[i] = SpibRegs.SPIRXBUF; // readdata[0] is garbage
665         }
666
667         PostSWI1(); // Manually cause the interrupt for the SWI1
668
669     } else if (CurrentChip == DAN28027) {
670
671         DAN28027Garbage = SpibRegs.SPIRXBUF;
672         dan28027adc1 = SpibRegs.SPIRXBUF;
673         dan28027adc2 = SpibRegs.SPIRXBUF;
674         CurrentChip = MPU9250;
675         SpibRegs.SPIFFCT.bit.TXDLY = 0;
676     }
677
678     SpibRegs.SPIFFRX.bit.RXFFOVFCLR=1; // Clear Overflow flag
679     SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
680     PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
681     GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
682 }

```



```

683
684
685 //adcd1 pie interrupt
686 __interrupt void ADCC_ISR (void)
687 {
688     GpioDataRegs.GPASET.bit.GPIO11 = 1;
689     adcc2result = AdccResultRegs.ADCRESULT0;
690     adcc3result = AdccResultRegs.ADCRESULT1;
691     adcc4result = AdccResultRegs.ADCRESULT2;
692     adcc5result = AdccResultRegs.ADCRESULT3;
693
694     // Here covert ADCIND0 to volts
695     adcC2Volt = adcc2result*3.0/4095.0;
696     adcC3Volt = adcc3result*3.0/4095.0;
697     adcC4Volt = adcc4result*3.0/4095.0;
698     adcC5Volt = adcc5result*3.0/4095.0;
699
700     if (MPU9250ignoreCNT >= 1) {
701         CurrentChip = MPU9250;
702         SpibRegs.SPIFFCT.bit.TXDLY = 0;
703         SpibRegs.SPIFFRX.bit.RXFFIL = 8;
704
705         GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
706
707         SpibRegs.SPITXBUF = ((0x8000)|(0x3A00));
708         SpibRegs.SPITXBUF = 0;
709         SpibRegs.SPITXBUF = 0;
710         SpibRegs.SPITXBUF = 0;
711         SpibRegs.SPITXBUF = 0;
712         SpibRegs.SPITXBUF = 0;
713         SpibRegs.SPITXBUF = 0;
714         SpibRegs.SPITXBUF = 0;
715     } else {
716         MPU9250ignoreCNT++;
717     }
718
719     numADCCcalls++;
720     AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
721     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
722     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
723 }
724 }
725
726 // cpu_timer0_isr - CPU Timer0 ISR
727 __interrupt void cpu_timer0_isr(void)
728 {
729     CpuTimer0.InterruptCount++;
730
731     numTimer0calls++;
732
733     if ((numTimer0calls%5) == 0) {
734         // Blink LaunchPad Red LED
735         //GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1;
736     }
737
738     // Acknowledge this interrupt to receive more interrupts from group 1
739     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
740 }
741
742 // cpu_timer1_isr - CPU Timer1 ISR
743 __interrupt void cpu_timer1_isr(void)
744 {
745     serial_sendSCIC(&SerialC, G_command, G_len);
746
747     CpuTimer1.InterruptCount++;
748 }
749 }
750
751 // cpu_timer2_isr CPU Timer2 ISR
752 __interrupt void cpu_timer2_isr(void)
753 {
754     // Blink LaunchPad Blue LED
755     //GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
756
757     CpuTimer2.InterruptCount++;
758
759     PostSWI3();
760
761     // if ((CpuTimer2.InterruptCount % 10) == 0) {
762     //     UARTPrint = 1;
763     // }
764 }
765 }
766
767 void setF28027EPWM1A(float controleffort){
768     if (controleffort < -10) {
769         controleffort = -10;
770     }
771     if (controleffort > 10) {
772         controleffort = 10;
773     }
774     float value = (controleffort+10)*3000.0/20.0;
775     EPwm1A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
776 }
777 void setF28027EPWM2A(float controleffort){
778     if (controleffort < -10) {
779         controleffort = -10;
780     }
781     if (controleffort > 10) {
782         controleffort = 10;
783     }
784     float value = (controleffort+10)*3000.0/20.0;
785     EPwm2A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
786 }
787
788 //
789 // Connected to PIEIER12_9 (use MINT12 and MG12_9 masks):
790 //
791 __interrupt void SWI1_HighestPriority(void) // EMIF_ERROR
792 {
793     GpioDataRegs.GPASET.bit.GPIO61 = 1;
794     // Set interrupt priority:
795     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
796     IER |= M_INT12;

```

```

797 IER    &= MINT12;                                // Set "global" priority
798 PieCtrlRegs.PIEIER12.all &= MG12_9; // Set "group" priority
799 PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
800 __asm(" NOP");
801 EINT;
802
803 //#####
804 IMU_data[0] = readdata[1];
805 IMU_data[1] = readdata[2];
806 IMU_data[2] = readdata[3];
807 IMU_data[3] = readdata[5];
808 IMU_data[4] = readdata[6];
809 IMU_data[5] = readdata[7];
810
811 accelx = (((float)(IMU_data[0]))*4.0/32767.0);
812 accely = (((float)(IMU_data[1]))*4.0/32767.0);
813 accelz = (((float)(IMU_data[2]))*4.0/32767.0);
814 gyrox = (((float)(IMU_data[3]))*250.0/32767.0);
815 gyroy = (((float)(IMU_data[4]))*250.0/32767.0);
816 gyroz = (((float)(IMU_data[5]))*250.0/32767.0);
817
818 if(calibration_state == 0){
819     calibration_count++;
820     if (calibration_count == 2000) {
821         calibration_state = 1;
822         calibration_count = 0;
823     }
824 } else if(calibration_state == 1){
825     accelx_offset+=accelx;
826     accely_offset+=accely;
827     accelz_offset+=accelz;
828     gyrox_offset+=gyrox;
829     gyroy_offset+=gyroy;
830     gyroz_offset+=gyroz;
831     gyroLPR510_offset+=adcC5Volt;
832     calibration_count++;
833     if (calibration_count == 2000) {
834         calibration_state = 2;
835         accelx_offset/=2000.0;
836         accely_offset/=2000.0;
837         accelz_offset/=2000.0;
838         gyrox_offset/=2000.0;
839         gyroy_offset/=2000.0;
840         gyroz_offset/=2000.0;
841         gyroLPR510_offset/=2000.0;
842         calibration_count = 0;
843     }
844 } else if(calibration_state == 2) {
845
846     gyroLPR510_drift += (((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005)/(2000);
847     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
848
849     gyro9250_drift += (((gyroz-gyro9250_offset) + old_gyro9250)*.0005)/(2000);
850     old_gyro9250 = gyro9250_offset;
851     calibration_count++;
852
853     if (calibration_count == 2000) {
854         calibration_state = 3;
855         calibration_count = 0;
856         doneCal = 1;
857         newOPTITRACKpose = 0;
858     }
859 } else if(calibration_state == 3){
860     accelx -=(accelx_offset);
861     accely -=(accely_offset);
862     accelz -=(accelz_offset);
863     gyrox -= gyrox_offset;
864     gyroy -= gyroy_offset;
865     gyroz -= gyroz_offset;
866     LeftWheel = readEncLeft();
867     RightWheel = readEncRight();
868     HandValue = readEncWheel();
869
870     gyro9250_angle = gyro9250_angle + (gyroz + old_gyro9250)*.0005 - gyro9250_drift;
871     old_gyro9250 = gyro9250_offset;
872
873     gyroLPR510_angle = gyroLPR510_angle + ((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005 - gyroLPR510_drift;
874     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
875
876     gyro9250_radians = (gyro9250_angle * (PI/180.0));
877     gyroLPR510_radians = gyroLPR510_angle * 400 * (PI/180.0);
878
879     LeftVel = (1.235/12.0)*(LeftWheel - LeftWheel_1)*1000;
880     RightVel = (1.235/12.0)*(RightWheel - RightWheel_1)*1000;
881
882     if (newLinuxCommands == 1) {
883         newLinuxCommands = 0;
884         printLinux1 = LinuxCommands[0];
885         printLinux2 = LinuxCommands[1];
886         ///? = LinuxCommands[2];
887         ///? = LinuxCommands[3];
888         ///? = LinuxCommands[4];
889         ///? = LinuxCommands[5];
890         ///? = LinuxCommands[6];
891         ///? = LinuxCommands[7];
892         ///? = LinuxCommands[8];
893         ///? = LinuxCommands[9];
894         ///? = LinuxCommands[10];
895     }
896
897     if (NewCAMDataThreshold1 == 1) {
898         NewCAMDataThreshold1 = 0;
899         MaxAreaThreshold1 = fromCAMvaluesThreshold1[0];
900         MaxColThreshold1 = fromCAMvaluesThreshold1[1];
901         MaxRowThreshold1 = fromCAMvaluesThreshold1[2];
902
903         NextLargestAreaThreshold1 = fromCAMvaluesThreshold1[3];
904         NextLargestColThreshold1 = fromCAMvaluesThreshold1[4];
905         NextLargestRowThreshold1 = fromCAMvaluesThreshold1[5];
906
907         NextNextLargestAreaThreshold1 = fromCAMvaluesThreshold1[6];
908         NextNextLargestColThreshold1 = fromCAMvaluesThreshold1[7];
909         NextNextLargestRowThreshold1 = fromCAMvaluesThreshold1[8];
910         numThres1++;

```



```

911         float mr = MaxRowThreshold1;
912
913         odist = -1.15291070692441e-05*pow(mr,3)+0.00348114123702629*pow(mr,2)-0.365334496447587*pow(mr,1)+13.7577551982332+0.75;
914         if ((numThres1 % 5) == 0) {
915             // LED4 is GPIO97
916             GpioDataRegs.GPDT0GGLE.bit.GPIO97 = 1;
917         }
918         if ((numThres1 % 100) == 0) {
919             UARTPrint = 1;
920         }
921     }
922
923     if (NewCAMDataThreshold2 == 1) {
924         NewCAMDataThreshold2 = 0;
925         MaxAreaThreshold2 = fromCAMvaluesThreshold2[0];
926         MaxColThreshold2 = fromCAMvaluesThreshold2[1];
927         MaxRowThreshold2 = fromCAMvaluesThreshold2[2];
928
929         NextLargestAreaThreshold2 = fromCAMvaluesThreshold2[3];
930         NextLargestColThreshold2 = fromCAMvaluesThreshold2[4];
931         NextLargestRowThreshold2 = fromCAMvaluesThreshold2[5];
932
933         NextNextLargestAreaThreshold2 = fromCAMvaluesThreshold2[6];
934         NextNextLargestColThreshold2 = fromCAMvaluesThreshold2[7];
935         NextNextLargestRowThreshold2 = fromCAMvaluesThreshold2[8];
936         numThres2++;
937         float mr = MaxRowThreshold2;
938
939         gdist = -1.15291070692441e-05*pow(mr,3)+0.00348114123702629*pow(mr,2)-0.365334496447587*pow(mr,1)+13.7577551982332+0.75;
940
941         if ((numThres2 % 5) == 0) {
942             // LED5 is GPIO111
943             GpioDataRegs.GPDT0GGLE.bit.GPIO111 = 1;
944         }
945         if ((numThres2 % 100) == 0) {
946             UARTPrint = 1;
947         }
948     }
949
950     if (NewCAMDataAprilTag1 == 1) {
951         NewCAMDataAprilTag1 = 0;
952         tagx = fromCAMvaluesAprilTag1[0];
953         tagy = fromCAMvaluesAprilTag1[1];
954         tagz = fromCAMvaluesAprilTag1[2];
955
956         tagthetax = fromCAMvaluesAprilTag1[3];
957         tagthetay = fromCAMvaluesAprilTag1[4];
958         tagthetaz = fromCAMvaluesAprilTag1[5];
959
960         tagid = fromCAMvaluesAprilTag1[6];
961
962         numtag++;
963         if ((numtag % 5) == 0) {
964             // LED2 is GPIO94
965             GpioDataRegs.GPCT0GGLE.bit.GPIO94 = 1;
966         }
967     }
968
969     if (NewLVData == 1) {
970         NewLVData = 0;
971         printLV1 = fromLVvalues[0];
972         printLV2 = fromLVvalues[1];
973         //?? = fromLVvalues[2];
974         //?? = fromLVvalues[3];
975         //?? = fromLVvalues[4];
976         //?? = fromLVvalues[5];
977         //?? = fromLVvalues[6];
978         //?? = fromLVvalues[7];
979     }
980
981
982     if (new_optitrack == 1) {
983         OPTITRACKps = UpdateOptitrackStates(ROBOTps, &newOPTITRACKpose);
984         new_optitrack = 0;
985     }
986
987     // Step 0: update B, u
988     B[0][0] = cosf(ROBOTps.theta)*0.001;
989     B[1][0] = sinf(ROBOTps.theta)*0.001;
990     B[2][1] = 0.001;
991     u[0][0] = 0.5*(LeftVel + RightVel); // linear velocity of robot
992     u[1][0] = gyro*(PI/180.0); // angular velocity in rad/s (negative for right hand angle)
993
994     // Step 1: predict the state and estimate covariance
995     Matrix3x2_Mult(B, u, Bu); // Bu = B*u
996     Matrix3x1_Add(x_pred, Bu, x_pred, 1.0, 1.0); // x_pred = x_pred(old) + Bu
997     Matrix3x3_Add(P_pred, Q, P_pred, 1.0, 1.0); // P_pred = P_pred(old) + Q
998     // Step 2: if there is a new measurement, then update the state
999     if (1 == newOPTITRACKpose) {
1000         OptiNumUpdatesProcessed++; // checking how often OptiTrack is causing a Kalman update
1001         if ((OptiNumUpdatesProcessed%10)==0) {
1002             // LED 3
1003             GpioDataRegs.GPCT0GGLE.bit.GPIO95 = 1;
1004         }
1005         newOPTITRACKpose = 0;
1006         z[0][0] = OPTITRACKps.x; // take in the Optitrack Motion Capture measurement
1007         z[1][0] = OPTITRACKps.y;
1008         // fix for OptiTrack problem at 180 degrees
1009         if (cosf(ROBOTps.theta) < -0.99) {
1010             z[2][0] = ROBOTps.theta;
1011         }
1012         else {
1013             z[2][0] = OPTITRACKps.theta;
1014         }
1015         // Step 2a: calculate the innovation/measurement residual, ytilde
1016         Matrix3x1_Add(z, x_pred, ytilde, 1.0, -1.0); // ytilde = z-x_pred
1017         // Step 2b: calculate innovation covariance, S
1018         Matrix3x3_Add(P_pred, R, S, 1.0, 1.0); // S = P_pred + R
1019         // Step 2c: calculate the optimal Kalman gain, K
1020         Matrix3x3_Invert(S, S_inv);
1021         Matrix3x3_Mult(P_pred, S_inv, K); // K = P_pred*(S^-1)
1022         // Step 2d: update the state estimate x_pred = x_pred(old) + K*ytilde
1023         Matrix3x1_Mult(K, ytilde, temp_3x1);
1024         Matrix3x1_Add(x_pred, temp_3x1, x_pred, 1.0, 1.0);

```

```

1025 // Step 2e: update the covariance estimate P_pred = (I-K)*P_pred(old)
1026 Matrix3x3 Add(eye3, K, temp_3x3, 1.0, -1.0);
1027 Matrix3x3_Mult(temp_3x3, P_pred, P_pred);
1028 } // end of correction step
1029
1030 // set ROBOTps to the updated and corrected Kalman values.
1031 ROBOTps.x = x_pred[0][0];
1032 ROBOTps.y = x_pred[1][0];
1033 ROBOTps.theta = x_pred[2][0];
1034
1035 // Make sure this function is called every time in this function even if you decide not to use its vref and turn
1036 // uses xy code to step through an array of positions
1037 if( xy_control(&vref, &turn, 1.0, ROBOTps.x, ROBOTps.y, robotdest[statePos].x, robotdest[statePos].y, ROBOTps.theta, 0.25, 0.5)) {
1038     statePos = (statePos+1)%NUMWAYPOINTS;
1039 }
1040 // state machine
1041 //RobotState = 20;
1042 switch (RobotState) {
1043 case 1:
1044
1045     // vref and turn are the vref and turn returned from xy_control
1046
1047     if (LADARfront < 1.2) {
1048         vref = 0.2;
1049         checkfronttally++;
1050         if (checkfronttally > 310) { // check if LADARfront < 1.2 for 310ms or 3 LADAR samples
1051             RobotState = 10; // Wall follow
1052             WallFollowtime = 0;
1053             right_wall_follow_state = 1;
1054         }
1055     } else {
1056         checkfronttally = 0;
1057     }
1058     //cjca cause a 2 second delay when resetting
1059     count1 += 1;
1060     if (count1 > 2000) {
1061         //cjca decide which colored ball is closer
1062         if (MaxAreaThreshold1 > 35 & (MaxAreaThreshold1 > MaxAreaThreshold2)) {
1063             RobotState=20;
1064             //cjca also checks threshold of other ball as well
1065         } else if (MaxAreaThreshold2 > 35 & (MaxAreaThreshold2 > MaxAreaThreshold1)) {
1066             RobotState=30;
1067         }
1068     }
1069     break;
1070 case 10:
1071     if (right_wall_follow_state == 1) {
1072         //Left Turn
1073         turn = Kp_front_wall*(14.5 - LADARfront);
1074         vref = front_turn_velocity;
1075         if (LADARfront > left_turn_Stop_threshold) {
1076             right_wall_follow_state = 2;
1077         }
1078     } else if (right_wall_follow_state == 2) {
1079         //Right Wall Follow
1080         turn = Kp_right_wal*(ref_right_wall - LADARrightfront);
1081         vref = foward_velocity;
1082         if (LADARfront < left_turn_Start_threshold) {
1083             right_wall_follow_state = 1;
1084         }
1085     }
1086     if (turn > turn_saturation) {
1087         turn = turn_saturation;
1088     }
1089     if (turn < -turn_saturation) {
1090         turn = -turn_saturation;
1091     }
1092
1093     WallFollowtime++;
1094     if ( (WallFollowtime > 5000) && (LADARfront > 1.5) ) {
1095         RobotState = 1;
1096         checkfronttally = 0;
1097     }
1098     break;
1099
1100 case 20:
1101     // put vision code here
1102     //cjca orange ball detected
1103     kpvision = -0.05;
1104     colcentroid = MaxColThreshold1 - 80;
1105     if (MaxColThreshold1 == 0 || MaxAreaThreshold1 < 3) {
1106         vref = 0;
1107         turn = 0;
1108     } else {
1109         vref = 0.75;
1110         turn = kpvision*(0-colcentroid);
1111     }
1112
1113     if (MaxRowThreshold1 > 115) {
1114         RobotState = 22;
1115         count22 = 0;
1116     }
1117     break;
1118 case 22:
1119     //cjca stop to let gripper door open
1120
1121     vref = 0;
1122     turn = 0;
1123     count22 += 1;
1124     if (count22 > 1000) {
1125         RobotState = 24;
1126         count22 = 0;
1127     }
1128     break;
1129 case 24:
1130     //cjca increment in states to pick up ball smoother
1131     vref = 0.5;
1132     turn = 0;
1133     count22 += 1;
1134     if (count22 > 1000) {
1135         RobotState = 26;
1136         count22 = 0;
1137     }
1138     break;

```

```

1139     case 26:
1140         vref = 0;
1141         turn = 0;
1142         count22 += 1;
1143         if (count22 > 1000) {
1144             RobotState = 1;
1145             count22 = 0;
1146             count1 = 0;
1147         }
1148         //cjca continue to x,y point
1149         break;
1150
1151
1152     case 30:
1153         // put vision code here
1154         //cjca do the same but for the green ball now
1155         kpvision = -0.05;
1156         colcentroid = MaxColThreshold2 - 80;
1157         if (MaxColThreshold2 == 0 || MaxAreaThreshold2 < 3) {
1158             vref = 0;
1159             turn = 0;
1160         } else {
1161             vref = 0.75;
1162             turn = kpvision*(0-colcentroid);
1163         }
1164
1165         if (MaxRowThreshold2 > 100) {
1166             RobotState = 32;
1167             count22 = 0;
1168         }
1169         break;
1170     ///cjca delay by a second each time to run smoother
1171     case 32:
1172         vref = 0;
1173         turn = 0;
1174         count22 += 1;
1175         if (count22 > 1000) {
1176             RobotState = 34;
1177             count22 = 0;
1178         }
1179         break;
1180     //cjca uses the same count variable for convenience
1181     case 34:
1182         vref = 0.5;
1183         turn = 0;
1184         count22 += 1;
1185         if (count22 > 1000) {
1186             RobotState = 36;
1187             count22 = 0;
1188         }
1189         break;
1190     case 36:
1191         vref = 0;
1192         turn = 0;
1193         count22 += 1;
1194         if (count22 > 1000) {
1195             RobotState = 1;
1196             count22 = 0;
1197             //cjca reset the count variable for 2 second delay
1198             count1 = 0;
1199         }
1200         break;
1201     default:
1202         break;
1203 }
1204
1205
1206 //Must be called each time into this SWI1 function
1207 PIcontrol(&uLeft,&uRight,vref,turn,LeftWheel,RightWheel);
1208 // These below lines also must be called each time into this SWI1 function
1209 setEPWM1A(uLeft);
1210 setEPWM2A(-uRight);
1211 setF28027EPWM1A(uLeft);
1212 setF28027EPWM2A(-uRight);
1213 LeftWheel_1 = LeftWheel;
1214 RightWheel_1 = RightWheel;
1215
1216 if((timecount%250) == 0) {
1217     DataToLabView.floatData[0] = ROBOTps.x;
1218     DataToLabView.floatData[1] = ROBOTps.y;
1219     DataToLabView.floatData[2] = ROBOTps.theta;
1220     DataToLabView.floatData[3] = (float)timecount;
1221     DataToLabView.floatData[4] = 0.5*(LeftVel + RightVel);
1222     DataToLabView.floatData[5] = (float)RobotState;
1223     DataToLabView.floatData[6] = (float)statePos;
1224     DataToLabView.floatData[7] = LADARfront;
1225     LVsenddata[0] = '*'; // header for LVdata
1226     LVsenddata[1] = '$';
1227     for (i=0;i<LVNUM_TOFROM_FLOATS*4;i++) {
1228         if (i%2==0) {
1229             LVsenddata[i+2] = DataToLabView.rawData[i/2] & 0xFF;
1230         } else {
1231             LVsenddata[i+2] = (DataToLabView.rawData[i/2]>>8) & 0xFF;
1232         }
1233     }
1234     serial_sendSCID(&SerialID, LVsenddata, 4*LVNUM_TOFROM_FLOATS + 2);
1235 }
1236
1237 }
1238 timecount++;
1239 if((timecount%200) == 0)
1240 {
1241     if(doneCal == 0) {
1242         GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1; // Blink Blue LED while calibrating
1243     }
1244     GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1; // Always Block Red LED
1245     UARTPrint = 1; // Tell While loop to print
1246 }
1247
1248 SpibRegs.SPIFFCT.bit.TXDLY = 16;
1249 CurrentChip = DAN28027;
1250 SpibRegs.SPIFFRX.bit.RXFFIL = 3;
1251 GpioDataRegs.GPACLEAR.bit.GPIO29 = 1;
1252 SpibRegs.SPITXBUF = 0x00DA;

```

```

1253 SpibRegs.SPITXBUF = EPwm1A_F28027;
1254 SpibRegs.SPITXBUF = EPwm2A_F28027;
1255
1256
1257 //#####
1258 //
1259 // Restore registers saved:
1260 //
1261 DINT;
1262 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1263
1264 GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
1265 }
1266
1267 //
1268 // Connected to PIEIER12_10 (use MINT12 and MG12_10 masks):
1269 //
1270 __interrupt void SWI2_MiddlePriority(void) // RAM_CORRECTABLE_ERROR
1271 {
1272 // Set interrupt priority:
1273 volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1274 IER |= M_INT12;
1275 IER &= MINT12; // Set "global" priority
1276 PieCtrlRegs.PIEIER12.all &= MG12_10; // Set "group" priority
1277 PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
1278 __asm(" NOP");
1279 EINT;
1280
1281 //#####
1282 // Insert SWI ISR Code here.....
1283 // LED1
1284 GpioDataRegs.GPATOGGLE.bit.GPIO22 = 1;
1285 if (LADARpingpong == 1) {
1286 // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1287 LADARrightfront = 19; // 19 is greater than max feet
1288 for (LADARi = 52; LADARi <= 56; LADARi++) {
1289 if (ladar_data[LADARi].distance_ping < LADARrightfront) {
1290 LADARrightfront = ladar_data[LADARi].distance_ping;
1291 }
1292 }
1293 // LADARfront is the min of dist 111, 112, 113, 114, 115
1294 LADARfront = 19;
1295 for (LADARi = 111; LADARi <= 115; LADARi++) {
1296 if (ladar_data[LADARi].distance_ping < LADARfront) {
1297 LADARfront = ladar_data[LADARi].distance_ping;
1298 }
1299 }
1300 LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta));
1301 LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)+LADARps.y*cosf(ROBOTps.theta));
1302 for (LADARi = 0; LADARi < 228; LADARi++) {
1303
1304 ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_ping*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1305 ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_ping*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1306
1307 }
1308 } else if (LADARpingpong == 0) {
1309 // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1310 LADARrightfront = 19; // 19 is greater than max feet
1311 for (LADARi = 52; LADARi <= 56; LADARi++) {
1312 if (ladar_data[LADARi].distance_pong < LADARrightfront) {
1313 LADARrightfront = ladar_data[LADARi].distance_pong;
1314 }
1315 }
1316 // LADARfront is the min of dist 111, 112, 113, 114, 115
1317 LADARfront = 19;
1318 for (LADARi = 111; LADARi <= 115; LADARi++) {
1319 if (ladar_data[LADARi].distance_pong < LADARfront) {
1320 LADARfront = ladar_data[LADARi].distance_pong;
1321 }
1322 }
1323 LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta - PI/2.0));
1324 LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-LADARps.y*cosf(ROBOTps.theta - PI/2.0));
1325 for (LADARi = 0; LADARi < 228; LADARi++) {
1326
1327 ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_pong*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1328 ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_pong*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1329
1330 }
1331 }
1332
1333
1334
1335
1336 //#####
1337 //
1338 // Restore registers saved:
1339 //
1340 DINT;
1341 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1342
1343 }
1344
1345 //
1346 // Connected to PIEIER12_11 (use MINT12 and MG12_11 masks):
1347 //
1348 __interrupt void SWI3_LowestPriority(void) // FLASH_CORRECTABLE_ERROR
1349 {
1350 // Set interrupt priority:
1351 volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1352 IER |= M_INT12;
1353 IER &= MINT12; // Set "global" priority
1354 PieCtrlRegs.PIEIER12.all &= MG12_11; // Set "group" priority
1355 PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
1356 __asm(" NOP");
1357 EINT;
1358
1359 //#####
1360 // Insert SWI ISR Code here.....
1361 //c/c enable the ewpm's for servo's
1362 RCangle = readEncWheel();
1363 setEPWM3A_RCServo(RCangle); //RCangle is a value from -90 to 90
1364 setEPWM3B_RCServo(RCangle); //RCangle is a value from -90 to 90
1365 setEPWM5A_RCServo(RCangle); //RCangle is a value from -90 to 90
1366 setEPWM5B_RCServo(RCangle); //RCangle is a value from -90 to 90

```

```

1367     setEPWM6A_RCServo(RCangle); //RCangle is a value from -90 to 90
1368
1369     ////////////////////////////////////
1370     //
1371     // Restore registers saved:
1372     //
1373     DINT;
1374     PieCtrlRegs.PIEIER12.all = TempPIEIER;
1375
1376 }
1377
1378 // ----- code for CAN start here -----
1379 __interrupt void can_isr(void)
1380 {
1381     int i = 0;
1382
1383     uint32_t status;
1384
1385     //
1386     // Read the CAN interrupt status to find the cause of the interrupt
1387     //
1388     status = CANGetInterruptCause(CANB_BASE);
1389
1390     //
1391     // If the cause is a controller status interrupt, then get the status
1392     //
1393     if(status == CAN_INT_INT0ID_STATUS)
1394     {
1395         //
1396         // Read the controller status. This will return a field of status
1397         // error bits that can indicate various errors. Error processing
1398         // is not done in this example for simplicity. Refer to the
1399         // API documentation for details about the error status bits.
1400         // The act of reading this status will clear the interrupt.
1401         //
1402         status = CANgetStatus(CANB_BASE);
1403
1404     }
1405
1406     //
1407     // Check if the cause is the receive message object 2
1408     //
1409     else if(status == RX_MSG_OBJ_ID_1)
1410     {
1411         //
1412         // Get the received message
1413         //
1414         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_1, rxMsgData);
1415
1416         for(i = 0; i<2; i++)
1417         {
1418             dis_raw_1[i] = rxMsgData[i];
1419         }
1420
1421         dis_1 = 256*dis_raw_1[1] + dis_raw_1[0];
1422
1423         measure_status_1 = rxMsgData[2];
1424
1425         //
1426         // Getting to this point means that the RX interrupt occurred on
1427         // message object 2, and the message RX is complete. Clear the
1428         // message object interrupt.
1429         //
1430         CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_1);
1431
1432         //
1433         // Increment a counter to keep track of how many messages have been
1434         // received. In a real application this could be used to set flags to
1435         // indicate when a message is received.
1436         //
1437         rxMsgCount_1++;
1438
1439         //
1440         // Since the message was received, clear any error flags.
1441         //
1442         errorFlag = 0;
1443     }
1444
1445     else if(status == RX_MSG_OBJ_ID_2)
1446     {
1447         //
1448         // Get the received message
1449         //
1450         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_2, rxMsgData);
1451
1452         for(i = 0; i<2; i++)
1453         {
1454             dis_raw_2[i] = rxMsgData[i];
1455         }
1456
1457         dis_2 = 256*dis_raw_2[1] + dis_raw_2[0];
1458
1459         measure_status_2 = rxMsgData[2];
1460
1461         //
1462         // Getting to this point means that the RX interrupt occurred on
1463         // message object 2, and the message RX is complete. Clear the
1464         // message object interrupt.
1465         //
1466         CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_2);
1467
1468         //
1469         // Increment a counter to keep track of how many messages have been
1470         // received. In a real application this could be used to set flags to
1471         // indicate when a message is received.
1472         //
1473         rxMsgCount_2++;
1474
1475         //
1476         // Since the message was received, clear any error flags.
1477         //
1478         errorFlag = 0;
1479     }
1480

```

```

1481 else if(status == RX_MSG_OBJ_ID_3)
1482 {
1483     //
1484     // Get the received message
1485     //
1486     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_3, rxMsgData);
1487
1488     for(i = 0; i<4; i++)
1489     {
1490         lightlevel_raw_1[i] = rxMsgData[i];
1491         quality_raw_1[i] = rxMsgData[i+4];
1492     }
1493
1494     lightlevel_1 = ((256.0*256.0*256.0)*lightlevel_raw_1[3] + (256.0*256.0)*lightlevel_raw_1[2] + 256.0*lightlevel_raw_1[1] + lightlevel_raw_1[0])/65535;
1495     quality_1 = ((256.0*256.0*256.0)*quality_raw_1[3] + (256.0*256.0)*quality_raw_1[2] + 256.0*quality_raw_1[1] + quality_raw_1[0])/65535;
1496
1497
1498
1499     //
1500     // Getting to this point means that the RX interrupt occurred on
1501     // message object 2, and the message RX is complete. Clear the
1502     // message object interrupt.
1503     //
1504     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_3);
1505
1506     //
1507     // Since the message was received, clear any error flags.
1508     //
1509     errorFlag = 0;
1510 }
1511
1512
1513 else if(status == RX_MSG_OBJ_ID_4)
1514 {
1515     //
1516     // Get the received message
1517     //
1518     CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_4, rxMsgData);
1519
1520     for(i = 0; i<4; i++)
1521     {
1522         lightlevel_raw_2[i] = rxMsgData[i];
1523         quality_raw_2[i] = rxMsgData[i+4];
1524     }
1525
1526     lightlevel_2 = ((256.0*256.0*256.0)*lightlevel_raw_2[3] + (256.0*256.0)*lightlevel_raw_2[2] + 256.0*lightlevel_raw_2[1] + lightlevel_raw_2[0])/65535;
1527     quality_2 = ((256.0*256.0*256.0)*quality_raw_2[3] + (256.0*256.0)*quality_raw_2[2] + 256.0*quality_raw_2[1] + quality_raw_2[0])/65535;
1528
1529
1530
1531     //
1532     // Getting to this point means that the RX interrupt occurred on
1533     // message object 2, and the message RX is complete. Clear the
1534     // message object interrupt.
1535     //
1536     CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_4);
1537
1538     //
1539     // Since the message was received, clear any error flags.
1540     //
1541     errorFlag = 0;
1542 }
1543
1544
1545 //
1546 // If something unexpected caused the interrupt, this would handle it.
1547 //
1548 else
1549 {
1550     //
1551     // Spurious interrupt handling can go here.
1552     //
1553 }
1554
1555 //
1556 // Clear the global interrupt flag for the CAN interrupt line
1557 //
1558 CANclearGlobalInterruptStatus(CANB_BASE, CAN_GLOBAL_INT_CANINT0);
1559
1560 //
1561 // Acknowledge this interrupt located in group 9
1562 //
1563 InterruptclearACKGroup(INTERRUPT_ACK_GROUP9);
1564 }
1565 // ----- code for CAN end here -----

```